

CS 5326 · PA1: The Modern Transformer LM

Quiz Study Guide: concepts, reasons, your implementation, and a full practice quiz with answers

Part	What it contains	How to use it
A	Section-by-section notes for PA1 §1–§7: every key term, the formula, why it is used, and how your code does it.	Read once end to end. The yellow “ Why ” boxes are the most likely quiz material.
B	Your approach and results: tuning ladder, final configuration, standardized evaluation, decoding study, and the bugs found in review.	Be ready to explain every choice you made and what the evidence was.
C	One-page formula and numbers cheat sheet.	Revise the night before and just before the quiz.
D	Practice quiz: 133 questions (concept, calculation, code-reading, “why”).	Answer on paper first, without looking at Part E.
E	Answer key with short explanations.	Mark yourself; re-read the Part A section for anything you missed.

Source of truth. Everything here follows PA1.pdf (the assignment manual), your submitted code in `src/`, your `REPORT.md`, and numbers re-measured from your `final_model.pt` and `tuning/results.csv`. Section numbers (§) refer to PA1.pdf.

Your headline numbers. 19,272,192 parameters · 10,000 optimizer updates · 65,536 tokens per update · 655,360,000 sampled tokens · standardized validation cross-entropy **1.6114** nats/token · perplexity **5.0100**.

Part A — Concepts, section by section

§1 Assignment overview and the rules

You build a **decoder-only (GPT-style) Transformer language model** from scratch, train it on TinyStories, measure its perplexity, and study how decoding settings change its generations. A language model maps token IDs of shape (batch, seq) to **logits** of shape (batch, seq, vocab): at each position, an unnormalized score for every possible next token.

Key term	What it is	Why we use it / why it matters
Decoder-only LM	A stack of causal self-attention blocks that predicts token $i+1$ from tokens $1 \dots i$.	One model both trains (all positions in parallel) and generates (one token at a time).
Autoregressive	Generate a token, append it, feed the longer sequence back in.	Text is produced left-to-right; each step conditions on everything generated so far.
Logits	Unnormalized scores $z \in \mathbb{R}^{ V }$; $\text{softmax}(z)$ gives probabilities.	The model returns logits, not probabilities; softmax is applied only in the loss or when sampling.
From-scratch rule	No <code>torch.nn</code> / <code>torch.nn.functional</code> / <code>torch.optim</code> definitions except the listed exceptions.	The point is to understand and implement every component yourself.
Allowed exceptions	<code>nn.Parameter</code> ; container classes (<code>Module</code> , <code>ModuleList</code> , <code>Sequential</code>); <code>nn.init</code> (incl. <code>trunc_normal_</code>); <code>torch.optim.Optimizer</code> base class.	You still need parameters, module bookkeeping, initialization helpers, and the optimizer API.
Deliverable rule	If a function is itself a deliverable, you may not call its built-in equivalent.	E.g. no <code>torch.softmax</code> inside attention, no <code>F.cross_entropy</code> , no <code>torch.optim.AdamW</code> .
<code>tests/adapters.py</code>	The only test-harness file you edit. Each adapter imports your code, loads supplied weights, calls it.	Glue only: no maths in adapters. Grading (public + hidden tests) goes through the same adapters.
Grading	70% unit tests (public + hidden), 30% <code>REPORT.md</code> .	Hidden tests check the manual's contract, not just the public tests: e.g. your RoPE dtype bug passed public tests but broke the written contract.

Why hidden tests matter

Public tests exercise only part of each contract. Hidden tests may check the other written requirements, such as dtype preservation, rejection of bad inputs, and edge cases like `warmup = 0` or the last valid batch window. Always implement what the manual says, not just what the public tests check.

§2 Repository, environment, dataset and tokenizer

Key term	What it is	Why we use it / why it matters
<code>uv / uv sync --frozen</code>	Package manager; installs exactly the versions in <code>uv.lock</code> .	Reproducible, identical environment for every student and the grader.
<code>uv run pytest</code>	Runs the tests inside the project environment.	Keeps imports of <code>tests/adapters.py</code> consistent. Run the full suite (no <code>-k</code>) before submitting.
<code>pytest -k / node ID</code>	<code>-k</code> selects tests by name expression; <code>path::test_name</code> runs one exact test.	Fast iteration on one component.
Byte-level BPE	Tokenizer over UTF-8 bytes; frequent byte sequences are merged into tokens. Vocabulary 8,192.	No unknown-token problem (any byte sequence is representable) and shorter sequences than raw bytes.
EOT <code>< endoftext ></code>	Special token separating stories in the stream (id 0 in this tokenizer).	Marks document boundaries so a sampled window can cross stories; in generation, it signals the end of a story.
Pre-tokenized stream	<code>train.bin / validation.bin</code> : flat little-endian uint16 token IDs, no header.	No tokenization cost during training; do not decode and re-encode it.

Key term	What it is	Why we use it / why it matters
Dataset sizes	Train: 466,876,982 tokens (2,119,719 stories). Validation: 4,692,376 tokens (21,990 stories).	Needed for the Chinchilla / tokens-per-parameter reasoning in §7.5.
Not pretrained	The embedding table and LM head are ordinary parameters that start random.	Only the tokenizer is fixed; everything the model knows it learns in training.

§3 Transformer LM architecture

3.0 Fixed configuration (memorize this table)

Symbol	Variable	Meaning	Value
$ V $	vocab_size	vocabulary size	8192
$n_{\max}$	context_length	maximum sequence length	256
d_{model}	d_model	residual-stream width	512
L	num_layers	Transformer blocks	4
h_q	n_q_heads	query heads	16
h_{kv}	n_kv_heads	key/value heads	4
$g = h_q/h_{kv}$	group_size	query heads per K/V head	4
$d_h = d_{\text{model}}/h_q$	head_dim	attention head width	32
d_{ff}	d_ff	SwiGLU hidden width	1344
Θ	rope_theta	RoPE frequency base	10,000
ϵ_{norm}	norm_eps	RMSNorm constant	1e-5

Other fixed choices: **untied** embedding and LM head, **every projection bias-free**, **no dropout** anywhere, **no absolute position embeddings** (RoPE supplies position inside attention), and **pre-norm** blocks with a final RMSNorm.

3.1 Data flow

```
logits = lm_head( final_norm( block_L( ... block_1( token_embedding(ids) ) ) ) )

block(x):  x = x + attention( attention_norm(x), token_positions )
           x = x + swiglu( ffn_norm(x) )
```

3.2 Tensor conventions: $y = xW^T$

PyTorch stores a weight as $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, and features sit on the **last** axis. For a row-vector input, the code applies $y = xW^T$; the maths in the manual writes the same map as $y = Wx$. Leading dimensions are batch-like (batch, sequence, heads) and just broadcast. Tests inject weights in exactly this orientation, so storing W^T instead of W breaks every test. einops.einsum / rearrange name axes explicitly; jaxtyping annotations (e.g. `Float[Tensor, "batch seq d_model"]`) document shapes.

3.3 Initialization

Key term	What it is	Why we use it / why it matters
Linear init	$N(0, \sigma^2)$ with $\sigma^2 = 2/(d_{\text{in}} + d_{\text{out}})$, truncated at $\pm 3\sigma$ (nn.init.trunc_normal_).	Keeps activation and gradient variance roughly constant through layers (Xavier/Glorot-style); truncation removes rare huge weights.
Embedding init	$N(0, 1)$ truncated to $[-3, 3]$.	Unit-scale token vectors; RMSNorm rescales them anyway.
RMSNorm gain	Initialized to 1.	Starts as a pure normalization; the model learns per-feature scales.

Example: the SwiGLU gate projection 512→1344 has $\sigma = \sqrt{2/1856} \approx 0.0328$, so weights lie in about ± 0.098 .

3.4 Linear and Embedding modules

- **Linear:** nn.Parameter of shape (out, in); forward `x @ self.weight.T`; accepts (... , in) and returns (... , out); no bias. Your code: `src/linear.py`.
- **Embedding:** table (num_embeddings, embedding_dim); forward `self.weight[token_ids]` (integer indexing), so any id shape (...) becomes (... , d). Indexing a parameter is still differentiable for the selected rows. Your code: `src/embedding.py`.

3.5 RMSNorm

$$\text{RMSNorm}(x)_\ell = y_\ell \cdot x_\ell / \text{RMS}(x), \text{RMS}(x) = \sqrt{(1/d) \sum_c x_c^2 + \epsilon}$$

- Normalizes **only the last dimension**; no mean subtraction (no centering) and no bias, unlike LayerNorm.
- **Precision rule:** if the input is fp16/bf16, upcast to fp32, normalize, then cast the result back to the input dtype. fp32/fp64 inputs are kept as they are.

Why RMSNorm, and why upcast?

RMSNorm keeps LayerNorm's useful property (re-scaling activations to a stable size) with fewer operations. Empirically, the re-centering step is not needed (LLaMA). The upcast matters because squaring in fp16 overflows: the fp16 maximum is 65,504, so an activation of 300 already gives $300^2 = 90,000 \rightarrow \text{inf}$. The mean of many squares also loses precision in low-precision formats. Casting back keeps the module's dtype contract.

3.6 Pre-norm vs post-norm

Post-norm (original Transformer): $x = \text{Norm}(x + \text{Sublayer}(x))$. Pre-norm (used here): $x = x + \text{Sublayer}(\text{Norm}(x))$, with one extra final norm after the last block. In pre-norm there is a **clean residual path** from the embedding to the output with no normalization on it, which improves gradient flow and training stability; it is the standard in modern LMs. Each block has **two separate RMSNorms** (attention_norm, ffn_norm) with independent parameters.

3.7 SiLU and SwiGLU feed-forward network

$$\text{SiLU}(x) = x \cdot \sigma(x) = x / (1 + e^{-x})$$

$$\text{SwiGLU}(x) = W_{\text{down}} [\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x], W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{ff}} \times d}, W_{\text{down}} \in \mathbb{R}^{d \times d_{\text{ff}}}$$

- SiLU is smooth at 0 (unlike ReLU), allows small negative outputs, and $\text{SiLU}(0) = 0$. You may use `torch.sigmoid`, but not a built-in SiLU.
- **GLU (gated linear unit):** an element-wise product of a gated branch and a linear branch. The gate learns what to pass through, and the linear branch gives gradients a direct path.
- **Why $d_{\text{ff}} = 1344$?** Canonically $d_{\text{ff}} = (8/3) \cdot d_{\text{model}} = 1365.3$. Three matrices of this width have about the same parameter count as a classic 2-matrix FFN of width $4d$. Rounding to a multiple of 64 ($21 \times 64 = 1344$) is for hardware efficiency.
- FFN parameters per layer: $3 \times 512 \times 1344 = 2,064,384$. Your code: `src/silu.py`, `src/swiglu.py` (gate, up, down).

3.8 Rotary Position Embeddings (RoPE): adjacent-pair convention

RoPE injects position by **rotating** each 2-D coordinate pair of every query and key head by a position-dependent angle. Values are **not** rotated. RoPE has **no learnable parameters**.

$$\omega_k = \Theta^{-(2k-2)/d_h}, \varphi_{i,k} = i \cdot \omega_k, k = 1 \dots d_h/2$$

$$x'_{2k-1} = x_{2k-1} \cos \varphi - x_{2k} \sin \varphi, x'_{2k} = x_{2k-1} \sin \varphi + x_{2k} \cos \varphi$$

$$\text{Efficient form: } R_i x = x \odot c_i + \text{rotate_pair}(x) \odot s_i, \text{rotate_pair}(x) = (-x_2, x_1, -x_4, x_3, \dots)$$

- **Relative-position property:** $R(\varphi_i)^T R(\varphi_j) = R((j-i)\omega)$, so $(R_i q) \cdot (R_j k) = q^T R_{j-i} k$. The attention score depends on the **offset $j-i$** , not the absolute positions.
- **Adjacent pairs** $(x_1, x_2), (x_3, x_4), \dots$ vs the GPT-NeoX **half-split** convention $(x_k, x_{k+d/2})$. Both are valid RoPE, but they are different coordinate layouts that must not be mixed. The tests use adjacent pairs.

- For $d_h = 32$ there are 16 frequencies: $\omega_1 = 1$ (fastest rotation) down to $\omega_{16} = 10000^{-30/32} \approx 1.78 \times 10^{-4}$ (slowest). This gives a spectrum from fine to coarse position information.
- **Buffers:** precompute cos and sin tables of shape $(\text{context_length}, d_h/2) = (256, 16)$ and register them with `persistent=False`.
- **Rejections required:** odd `head_dim`, wrong final dimension, non-integer positions, positions outside $[0, \text{context_length})$.
- **Output dtype must equal input dtype** (bf16 in $\rightarrow$ bf16 out). Your first version returned fp32 for bf16/fp16 because the fp32 buffers promoted the result. This was fixed by computing in fp32 and casting back.

Why buffers, why non-persistent, why compact, why no dense matrix?

Buffers move automatically with `model.to(device)` and are not trainable. Non-persistent means they are not saved in `state_dict`: they can be recomputed exactly from Θ and d_h , and the saved model then holds exactly the 19,272,192 learned values. The compact $d_h/2$ tables broadcast over the two coordinates of each pair, so there is no duplication. A dense $d_h \times d_h$ rotation matrix would be almost all zeros (block-diagonal), costing $O(d_h^2)$ instead of $O(d_h)$.

3.9 Softmax and scaled dot-product attention

$$\text{softmax}(z)_\ell = \exp(z_\ell - z_{\max}) / \sum_c \exp(z_c - z_{\max})$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V, Q: (\dots, n_q, d_k), K: (\dots, n_{kv}, d_k), V: (\dots, n_{kv}, d_v) \rightarrow (\dots, n_q, d_v)$$

- **Max-subtraction** leaves softmax unchanged (it is invariant to adding a constant) but prevents exp overflow: $\exp(1000) = \text{inf}$, and $\text{inf}/\text{inf} = \text{NaN}$.
- **Why $1/\sqrt{d_k}$?** If the components of q and k are independent with unit variance, $q \cdot k$ has variance d_k . Large scores push softmax into a near one-hot saturated region with tiny gradients, and scaling restores unit variance.
- **Mask convention:** True = query i may attend to key j . False entries are set to $-\infty$ before softmax, which gives them probability 0. The mask's leading dimensions broadcast against the score tensor.
- **Validation:** if every key is masked for some query, the row becomes all $-\infty$ and softmax gives NaN, so the spec says **fail clearly** (raise) instead. Your code checks `mask.sum(-1) == 0`.
- Leading dimensions broadcast, n_q may differ from n_{kv} , and d_v may differ from d_k .

3.10 Causal grouped-query self-attention (GQA)

Multi-head attention (MHA) runs h_q heads in parallel and concatenates them. **GQA** keeps $h_q = 16$ query heads but only $h_{kv} = 4$ key heads and 4 value heads; each K/V head is **shared by a group of $g = 4$ query heads**.

Tensor	Meaning	Shape
Q	grouped query heads (after RoPE)	$[B, h_{kv}, g, n, d_h]$
K, V	key / value heads (K after RoPE)	$[B, h_{kv}, n, d_h]$
S, A	scores / attention probabilities	$[B, h_{kv}, g, n, n]$
O	grouped output	$[B, h_{kv}, g, n, d_h] \rightarrow [B, n, h_q d_h] \rightarrow W_O$

- **Projections:** $W_Q (h_q d_h \times d) = 512 \times 512$, $W_K, W_V (h_{kv} d_h \times d) = 128 \times 512$, $W_O (d \times h_q d_h) = 512 \times 512$. **One matrix multiplication each** for all Q heads, all K heads, and all V heads.
- **Head order:** query head $a = (h-1)g + r$ belongs to K/V head h (1-indexed), with $r = 1 \dots g$. So Q heads 1–4 share K/V head 1, heads 5–8 share head 2, and so on. After attention, merge (h, r) back in the same order before W_O .
- **No repeated K/V:** compute scores with `einsum("bhrid, bhjd->bhrij")`. K_h is broadcast across the g query heads. Do not use `repeat_interleave`, which would copy K and V g times.
- **Causal mask:** key j is allowed iff $j \leq i$ (lower-triangular, `torch.tril`). This stops positions from seeing future tokens.
- The same code covers **MHA** ($h_{kv} = h_q, g = 1$), **GQA** ($1 < h_{kv} < h_q$), and **MQA** ($h_{kv} = 1, g = h_q$).

- Default positions are $0 \dots n-1$ when `token_positions` is `None`. Supplied positions are used for both Q and K, with shape $(n,)$ or (B, n) .

Why GQA?

Fewer K/V heads means fewer K/V parameters (this layer has 655,360 attention parameters instead of 1,048,576 for MHA). Most importantly, at inference the **KV cache is 4× smaller** (4 K/V heads instead of 16), which saves memory and memory bandwidth, while quality stays close to MHA. It interpolates between MHA (best quality, largest cache) and MQA (smallest cache, some quality loss).

3.11 Transformer block and full LM

- **Block:** $x \leftarrow x + \text{Attn}(\text{RMSNorm}_{\text{attn}}(x), \text{positions}); x \leftarrow x + \text{SwiGLU}(\text{RMSNorm}_{\text{ffn}}(x))$. Positions go to attention only, because the FFN is position-wise.
- **LM:** embedding $\rightarrow$ 4 blocks $\rightarrow$ final RMSNorm $\rightarrow$ `lm_head` (untied, bias-free). It returns logits $(B, n, |V|)$ with no softmax inside.
- Store `self.context_length`. Require $1 \leq n \leq \text{context_length}$ and reject empty or over-long sequences (the RoPE tables only have 256 rows).
- Do not hard-code dimensions: the tests use small models.

3.12 Parameter count = 19,272,192

Component	Formula	Count
Token embedding	$ V \cdot d = 8192 \cdot 512$	4,194,304
LM head (untied)	$ V \cdot d$	4,194,304
$W_Q + W_O$ per layer	$2 d^2 = 2 \cdot 512^2$	524,288
$W_K + W_V$ per layer	$2 d h_{kv} d_h = 2 \cdot 512 \cdot 128$	131,072
SwiGLU per layer	$3 d d_{ff} = 3 \cdot 512 \cdot 1344$	2,064,384
2 RMSNorms per layer	$2 d$	1,024
Per layer	sum of the four rows above	2,720,768
4 layers	$4 \times 2,720,768$	10,883,072
Final RMSNorm	d	512
Total	$2 V d + L[2d^2 + 2d h_{kv} d_h + 3d d_{ff} + 2d] + d$	19,272,192

Note that 43.5% of the parameters are in the embedding and head (vocabulary-dependent), and about 43% are in the SwiGLU FFNs.

§4 Loss and optimization

4.1 Cross-entropy and perplexity

$\ell(z, y) = -z_y + \log \sum_c e^{z_c}$ (log-sum-exp computed with a max shift), L = mean over all batch and sequence positions

perplexity = $\exp(\text{mean cross-entropy})$

- Select the target logit directly with `gather`; do **not** build a full `log(softmax(logits))` tensor. `torch.logsumexp` is allowed; `torch.softmax`, `torch.log_softmax` and `F.cross_entropy` are not.
- At initialization the model is roughly uniform: $\text{CE} \approx \ln 8192 = 9.01$ and $\text{PPL} \approx 8192$. Your smoke run logged 9.01 at step 1, which is a good sanity check.
- **Perplexity** = the effective number of equally likely choices. Your PPL of 5.01 means the model is on average as uncertain as a uniform choice among about 5 tokens. It does not literally assign equal probability to 5 tokens.
- **Average the CE first, then exponentiate.** Averaging per-batch perplexities overstates PPL, because by Jensen's inequality $\text{mean}(\exp) \geq \exp(\text{mean})$.

4.2 AdamW

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g; v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2$$

$$\hat{m} = m_t / (1-\beta_1^t); \hat{v} = v_t / (1-\beta_2^t)$$

$$\theta \leftarrow (1 - \alpha \lambda) \theta \text{ (decoupled weight decay); } \theta \leftarrow \theta - \alpha \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

Key term	What it is	Why we use it / why it matters
First moment m	Exponential moving average (EMA) of gradients.	Momentum: smooths noisy minibatch gradients.
Second moment v	EMA of squared gradients.	Per-parameter adaptive step size: parameters with large or variable gradients take smaller steps.
Bias correction	Divide by $1-\beta^t$.	m and v start at 0, so early estimates are biased toward 0. At $t = 1$, $\hat{m} = g$ and $\hat{v} = g^2$, giving an update of about $\alpha \cdot \text{sign}(g)$.
Decoupled weight decay	$\theta \leftarrow (1-\alpha\lambda)\theta$ applied directly, not added to the gradient.	With Adam + L2, the $\lambda\theta$ term is divided by $\sqrt{\hat{v}}$, so decay strength varies per parameter. AdamW keeps decay uniform and independent of gradient scale (Loshchilov & Hutter).
Per-parameter count t_p	state[p]["step"], incremented only when p actually has a gradient.	Parameters skipped on some steps must use their own count in the bias correction, not the global step s.
Lazy state	Create step = 0, exp_avg and exp_avg_sq (zeros_like(p)) on the first update with a gradient.	No memory for parameters that never receive gradients.
grad is None	Skip the parameter completely: no decay, no state, no step increment.	Required by the spec (e.g. frozen or unused parameters).
Sparse grads	Reject them.	The dense update equations do not apply.
Closure	If given, evaluate it once under torch.enable_grad() before any update, and return its loss.	The closure must be able to compute gradients even if the caller disabled grad.
torch.no_grad()	Run the whole update without building an autograd graph.	Otherwise, if a gradient carries a graph (create_graph=True), the moment buffers join the graph, leaking memory and gradients. This was your AdamW bug, now fixed with @torch.no_grad() on step.
Group validation	$lr \geq 0, \text{eps} \geq 0, \text{weight_decay} \geq 0, 0 \leq \beta_1, \beta_2 < 1$; checked in __init__ for every group and again inside step.	Callers (such as the LR scheduler) modify group["lr"] later, so invalid values must be rejected before any parameter is updated . You fixed a partial-update bug here.
state_dict	The inherited save/load of per-parameter steps and moments.	Exact continuation after resuming a checkpoint.

Memory note: fp32 parameters (77.1 MB) plus m and v (another 2×77.1 MB) come to about 231 MB, which is exactly the size of your full training checkpoints.

4.3 Cosine learning-rate schedule with linear warmup

$$\alpha_s = (s / s_w) \alpha_{\max} \text{ if } s < s_w$$

$$\alpha_s = \alpha_{\min} + \frac{1}{2} (1 + \cos(\pi (s - s_w) / (s_c - s_w))) (\alpha_{\max} - \alpha_{\min}) \text{ if } s_w \leq s \leq s_c$$

$$\alpha_s = \alpha_{\min} \text{ if } s > s_c$$

- Key values: $\alpha_0 = 0$ (with warmup), $\alpha_{s_w} = \alpha_{\max}$ (the peak is first reached at $s = s_w$), $\alpha_{s_c} = \alpha_{\min}$, and it stays at $\alpha_{\min}$ afterwards.
- Warmup = 0** is allowed: step 0 is already in the cosine branch, so $\alpha_0 = \alpha_{\max}$.

- Validation: $0 \leq \alpha_{\min} \leq \alpha_{\max}$; integer, non-negative steps; $s_w \geq 0$ and $s_w < s_c$. After the fix, bool is rejected (True is an int subclass) and NumPy integers are accepted.

Why warmup, why cosine, why hold at the floor?

Warmup: early on, Adam's second-moment estimates are based on few samples and the weights are random, so full-size steps can destabilize training. **Cosine decay:** large steps early for fast progress, then smoothly smaller steps late to settle into a low-loss region. **Hold at $\alpha_{\min}$:** cosine is periodic, so beyond s_c the formula would make the LR rise again.

4.4 Global gradient clipping

$\|g\|_2 = \sqrt{(\sum_p \sum_\ell (g_\ell^{(p)})^2)}$; if $\|g\| > M$: $g \leftarrow g \cdot M / (\|g\| + \epsilon_{\text{clip}})$, $\epsilon_{\text{clip}} = 1e-6$

- **One global norm and one scale factor** for all parameters. This preserves the direction of the full gradient and only shrinks its length.
- Ignore parameters whose gradient is None; return the **pre-clip** norm as a Python float (0.0 if there are no gradients); reject $M \leq 0$.
- Clipping happens **once per global step**: after all accumulated backward passes and before `optimizer.step()`.
- Your fix: square the gradients in at least fp32. In fp16, 300^2 overflows to inf, which made the norm inf and the scale 0, zeroing all gradients.

§5 Training infrastructure and loop

5.1 Data loader

- **np.memmap** maps the file into virtual memory, and the OS loads only the pages you touch. `train.bin` is 933.8 MB on disk; converting it to int64 would allocate about 3.7 GB, so never call `np.asarray(..., int64)` on the whole stream.
- Open it with `np.memmap(path, mode="r", dtype=np.dtype("<u2"))`: read-only, little-endian uint16 ($8192 < 2^{16} = 65,536$). Reject missing files and odd byte lengths (not a whole number of uint16 values).
- **Windows:** for a start `o`, read `n+1` tokens: `x = u[o : o+n]`, `y = u[o+1 : o+n+1]` (next-token targets, shifted by one).
- **Valid starts:** $0 \leq o < N - n$, i.e. `torch.randint(0, N - n, ...)`. The upper bound is exclusive, so the last valid window ends exactly at the end of the file. Using `N - n + 1` would read past the file.
- Slice the sampled windows while they are still backed by the memmap, stack only those, convert that small batch to `torch.long`, then move it to the device.
- Pass a **torch.Generator** to `randint`: reproducible batches without resetting the global seed. The same seed gives the same batch sequence.

5.2 Checkpointing

- Save: `model.state_dict()`, `optimizer.state_dict()`, **next_step**, `train_generator.get_state()`, `val_generator.get_state()`. Load restores all of them and returns `next_step`.
- **next_step** is the next global step to execute. After completing step `s`, save `s+1`; before any update it is 0. Resume with `for step in range(next_step, num_steps)`.
- Checkpoint only **between complete global steps**, because mid-accumulation gradients and a partially advanced generator are not saved.

5.3 Global steps, gradient accumulation, canonical order

$N_{\text{tok/update}} = B \cdot n \cdot n_{\text{acc}} \rightarrow 16 \cdot 256 \cdot 16 = 65,536$; effective batch = $B \cdot n_{\text{acc}} = 256$ sequences

- A **global step** is one optimizer update, not one forward pass or microbatch. The schedule, `next_step` and the 10,000-step budget all count optimizer updates.
- **Gradient accumulation:** `zero_grad` once, then for each of n_{acc} minibatches call `(loss / n_{\text{acc}}).backward()`. Dividing makes the accumulated gradient equal the gradient of the **mean** loss; without it the gradient is n_{acc}

times too large, which changes the effective LR and how clipping behaves.

- Log the **unscaled** mean training loss across the microbatches.

```
train mode -> lr = get_lr(step) -> zero_grad -> n_acc x (get_batch, forward, loss/n_acc, backward)
            -> clip ONCE -> optimizer.step ONCE -> completed = step + 1
            -> validate (if due) -> log (if due) -> checkpoint (if due) # final step forces all three
```

Why this exact order, and why two generators?

Validation comes before checkpointing so that the saved val_generator state already includes the batches sampled at that boundary, which makes resume exact. Separate train and validation generators mean that changing how often you validate does not change the sequence of training batches.

5.4 Validation: eval() vs no_grad vs inference_mode

Key term	What it is	Why we use it / why it matters
model.eval()	Switches modules to evaluation behaviour (e.g. dropout off).	It does not disable autograd. This model has no dropout, but the contract still requires it.
torch.no_grad()	Disables gradient recording.	Saves memory in evaluation.
torch.inference_mode()	Like no_grad, plus it skips extra autograd bookkeeping (version counters, view tracking).	Faster and lighter; required for validation and generation.
model.train(was_training)	Restores the previous mode.	Evaluation must not change the caller's state.
evaluate_validation	Samples exactly num_validation_batches batches from the validation stream with val_generator and returns the arithmetic mean CE.	A cheap, stable estimate of generalization during training.

Pre-flight check (the “Put it together” box): before the long run, a small model should (1) **overfit one fixed minibatch** (loss → near 0), (2) run validation under inference_mode, and (3) restore the previous mode. Re-checked on your code: loss 9.03 → 0.004 in 300 steps, and both checks pass.

§6 Generating text (decoding)

$$p(x_{i+1} = c \mid x_{1:i}) = \text{softmax}(z_i / \tau)_c \quad (z_i = \text{logits at the final position})$$

- A **non-empty prompt** is required, because you need a final position to get the first next-token logits.
- **Temperature $\tau > 0$** : $\tau < 1$ sharpens the distribution (more deterministic, more repetition); $\tau = 1$ leaves it unchanged; $\tau > 1$ flattens it (more random, more errors). As $\tau \rightarrow 0^+$ it approaches greedy argmax.
- **Top-p (nucleus), $0 < p \leq 1$** : sort the probabilities in descending order, keep the **shortest prefix whose cumulative sum is $\geq p$** , zero the rest, and renormalize. The candidate set **adapts**: it is small when the model is confident and large when it is uncertain.
- **Sampling**: torch.multinomial over the filtered sorted distribution (passing the generator), then map the sampled rank back through the sorted indices to get the real vocabulary ID.
- **Loop**: crop to the last context_length tokens → add a batch dimension → forward → take the last position's logits → temperature → softmax → top-p → sample → append. **Stop right after EOT** and keep it. Return prompt + completion as a 1-D tensor.
- **Validation**: 1-D long non-empty prompt, max_new_tokens ≥ 0 (0 returns the prompt), context_length > 0 and equal to model.context_length (so generation cannot silently crop to a different window), $\tau > 0$, $0 < p \leq 1$. Run under model.eval() and inference_mode, and restore the previous mode.
- **No KV cache**: every new token recomputes the whole cropped context (up to 256 tokens). This is simple but wasteful; a KV cache would reuse past keys and values.

§7 Training on TinyStories, evaluation, submission

Quantity	Value / rule
Updates	Exactly $N_{\text{updates}} = 10,000$, indexed $s = 0 \dots 9999$
Cosine endpoint	$s_c = 9999$, so the final update uses $\alpha_{9999} = \alpha_{\min}$
Sequence length	256 (fixed)
Recommended start	$B = 16$, $n_{\text{acc}} = 16$, $\alpha_{\max} = 3e-4$, $\alpha_{\min} = 3e-5$, $s_w = 200$, $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 1e-8$, $\lambda = 0.1$, $M = 1.0$
Sampled tokens	$10,000 \times 65,536 = 655,360,000 \approx 34.0$ tokens/parameter ≈ 1.40 corpus-equivalents (sampled with replacement, so these are not epochs)
Chinchilla heuristic	≈ 20 tokens/parameter $\rightarrow 19,272,192 \times 20 \approx 385.4$ M tokens; the corpus has 24.2 tokens/parameter
Short tuning runs	1,000–2,000 updates; same cap, init seed, train/val seeds, eval frequency and procedure; keep $s_c = 9999$
Standardized eval	Fresh generator, seed 42; 100 batches of 16×256 (409,600 tokens); PPL = $\exp(\text{mean CE})$
final_model.pt	Plain state_dict, all tensors on CPU, floating tensors cast to fp16, exactly 19,272,192 values; no optimizer or RNG state
Submission	bash make_submission.sh $\rightarrow$ submission.zip containing src/, tests/adapters.py, REPORT.md, report_assets/, final_model.pt; rename to <roll_number>_pa1.zip

- **Ablation ladder:** change one hyperparameter from the baseline, compare validation trajectories, keep the better-supported choice, repeat.
- **Why keep $s_c = 9999$ in short runs?** So each short run sees the same prefix of the LR schedule the full run will see. Compressing it to 1,500 would test a different schedule.
- **Effective-batch confound:** with a fixed step cap, a smaller effective batch also sees fewer tokens, so the runs are not equal-token comparisons.
- **Fresh seed-42 evaluation:** it makes the reported number independent of how often you validated during training, and comparable across submissions.
- **Why FP16 export?** Half the size (38.5 MB vs 77.1 MB) and fine for inference. Non-floating tensors keep their dtype.

Part B — Your approach and results

B1 Where each component lives in your code

File	What it implements (key idea)
linear.py / embedding.py	$x @ W.T$ with W (out, in); table indexing; trunc-normal initialization
rmsnorm.py	fp16/bf16 $\rightarrow$ fp32 upcast; $x / \sqrt{\text{mean}(x^2) + \text{eps}}$ $\cdot y$; cast back
silu.py / swiglu.py	$x \cdot \text{sigmoid}(x)$; $\text{down}(\text{silu}(\text{gate}(x))) * \text{up}(x)$
rope.py	fp32 cos/sin tables (ctx, $d/2$), non-persistent; adjacent-pair rotation via reshape to $(\dots, d/2, 2)$; input validation; output cast back to the input dtype
softmax.py	max-shifted exp / sum along dim
attention.py	scaled_dot_product_attention (mask checks, $-\text{inf}$ fill, stable softmax); GroupedQuerySelfAttention (q/k/v/output_proj, RoPE on Q and K, grouped einsum, tril causal mask, head merge)
transformer_block.py / transformer_lm.py	pre-norm block; embedding $\rightarrow$ blocks $\rightarrow$ final_norm $\rightarrow$ lm_head; length checks; self.context_length
loss.py	max-shifted log-sum-exp minus the gathered target logit, averaged
adamw.py	torch.optim.Optimizer subclass; group validation; @torch.no_grad step; lazy per-parameter state
schedule.py / gradient_clipping.py	warmup + cosine + floor with integer checks; global L2 clip in $\geq$ fp32, returns the pre-clip float
data.py / checkpoint.py	memmap loader; generator-driven get_batch; save/load of model, optimizer, next_step and both RNG states
train.py	argparse CLI; two generators (seed, seed+1); gradient accumulation; canonical validate $\rightarrow$ log $\rightarrow$ checkpoint order; resume
generation.py	temperature $\rightarrow$ stable softmax $\rightarrow$ sorted top-p $\rightarrow$ multinomial $\rightarrow$ map rank back $\rightarrow$ EOT stop; eval + inference_mode; restores mode

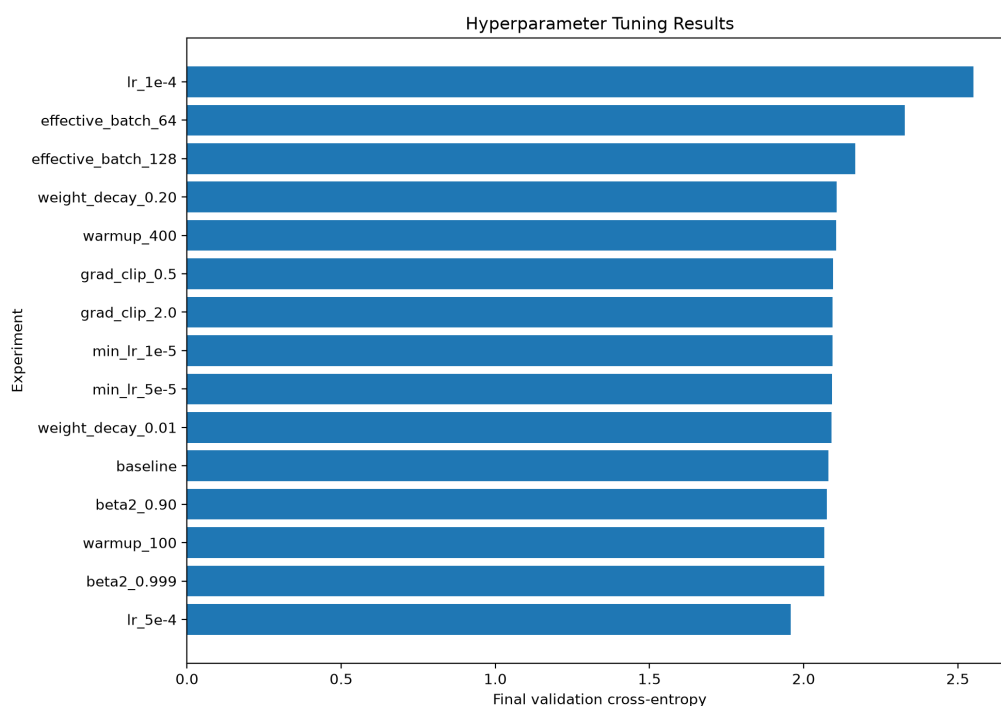
B2 Hyperparameter exploration (15 runs $\times$ 1,500 updates, $s_c = 9999$)

Run	Final val loss	Δ vs baseline	Final grad norm	Tokens
Baseline (recommended start)	2.0805	0	0.397	98.3 M
$\alpha_{\text{max}} = 1\text{e-}4$	2.5505	+0.4700	0.613	98.3 M
$\alpha_{\text{max}} = 5\text{e-}4$	1.9586	-0.1219	0.316	98.3 M
$\alpha_{\text{min}} = 1\text{e-}5 / 5\text{e-}5$	2.0934 / 2.0929	+0.0129 / +0.0124	0.431 / 0.452	98.3 M
Warmup 100 / 400	2.0669 / 2.1058	-0.0136 / +0.0253	0.415 / 0.451	98.3 M
$\beta_2 = 0.90 / 0.999$	2.0747 / 2.0667	-0.0058 / -0.0138	0.406 / 0.373	98.3 M
Weight decay 0.01 / 0.20	2.0907 / 2.1074	+0.0102 / +0.0269	0.417 / 0.426	98.3 M
Clip 0.5 / 2.0	2.0962 / 2.0943	+0.0157 / +0.0138	0.403 / 0.407	98.3 M
Effective batch 128 / 64	2.1675 / 2.3277	+0.0870 / +0.2472	0.452 / 0.510	49.2 M / 24.6 M

How to explain your reasoning (likely quiz questions)

- **Learning rate is the strongest effect** (-0.12 at $5\text{e-}4$, $+0.47$ at $1\text{e-}4$). With a fixed number of updates, a larger stable step size makes more progress. The $5\text{e-}4$ run did not diverge, and it had the lowest final gradient norm.
- **The minimum LR barely matters at 1,500 steps:** because $s_c = 9999$, the LR at step 1,500 is still about $2.88\text{e-}4$ in every run, so α_{min} hardly enters the formula yet.

- **Noise floor:** each configuration was run once, and model initialization was not seeded (train.py never calls torch.manual_seed; only the data generators are seeded, with 42 and 43). The two $\alpha_{\min}$ runs have almost the same LR as the baseline yet differ by about 0.013, so treat differences of about 0.02 or less as noise.
- Therefore **warmup 100** (−0.0136) and **β_2 0.999** (−0.0138) were adopted only as **weak-evidence, low-risk** choices. Weight decay and clipping changes were within noise, so the defaults were kept.
- **Effective batch:** the smaller-batch runs are confounded, because they also saw 2× and 4× fewer tokens. Under the fixed 10,000-update budget, the larger batch gives more tokens per update, so 256 was kept.
- **Limitation:** the three changes were never tested together in a short run, so their effects may not add up exactly.



Your report figure: final validation loss of each tuning run.

B3 Final run and evaluation

Setting	Final value
Batch / accumulation / effective batch	16 / 16 / 256 sequences (65,536 tokens per update)
$\alpha_{\max}$ / $\alpha_{\min}$ / warmup / s_c	5e-4 / 3e-5 / 100 / 9999
(β_1, β_2) , ϵ , weight decay, max grad norm	(0.9, 0.999), 1e-8, 0.1, 1.0 (confirmed from the final checkpoint's optimizer state)
Updates / tokens	10,000 / 655,360,000; every parameter's AdamW step count = 10,000
End of training	last 7 logged training losses: mean 1.5938 (sd 0.0186); gradient norm about 0.28, well below the clip threshold of 1.0
Training-time validation (last)	1.6437 (validation generator seed 43, already advanced by earlier evaluations)
Standardized evaluation	CE 1.6114 nats/token, PPL 5.0100 (seed 42, $100 \times 16 \times 256$); re-verified on the submitted fp16 final_model.pt

Why the two validation numbers differ (1.6437 vs 1.6114): they use different validation windows and a different number of batches, so the gap of about 0.03 is sampling variation. The standardized number, with 100 batches and a fresh seed, is the more reliable one. Training loss (about 1.59) is close to validation loss (about 1.61), so there is no obvious overfitting. That is expected, since the run sampled only about 1.4 corpus-equivalents.

B4 Decoding study (3 prompts × 5 seeds = 15 samples per setting)

Setting	Mean new tokens	Ended with EOT	Distinct-2	Repeated 4-grams
A: T = 0.7, p = 1.0	86.3	33%	0.868	2.0%
B: T = 1.0, p = 1.0 (reference)	97.7	13%	0.918	0.5%
C: T = 1.3, p = 1.0	83.0	33%	0.966	0.1%
D: T = 1.0, p = 0.8	85.9	20%	0.891	1.4%
E: T = 1.0, p = 0.9	89.3	27%	0.900	1.1%

- Prompts: “Once upon a time”, “The little girl”, “One day, a small”. They are typical TinyStories openings, open-ended, and the third stops mid-phrase to test grammatical completion.
- **Distinct-2** = unique token bigrams ÷ all bigrams (variety). **Repeated 4-grams** = the share of 4-grams already seen earlier in the same completion (looping).
- **Temperature**: as T rises, variety rises and repetition falls, but coherence drops. T = 1.3 produced non-words (“armgo”) and broken grammar, so a **higher distinct-2 is not better text**: variety metrics reward randomness as much as creativity.
- **Top-p**: lowering p reduces variety and increases repetition, like a milder version of lowering the temperature. It removed most surface errors in the samples shown, because rare tail tokens cause most mistakes.
- **Adaptive**: with the same seed, the p = 0.8 and p = 0.9 samples were identical for the first four sentences. Where the model is confident, both nuclei contain the same few tokens; differences appear only at uncertain steps.
- **Chosen balance**: T = 1.0, p = 0.9 (variety 0.900, near the untruncated 0.918; repetition 1.1%; grammatical with small logical slips). Coherence was judged by reading, so this is an observation, not a general law.

B5 Bugs found in the pre-submission review, and why they mattered

Issue	Why it was wrong	Fix
RoPE returned fp32 for bf16/fp16 input	The spec requires the same dtype as x; the fp32 buffers promoted the result.	Compute in promote_types(x.dtype, fp32), then .to(x.dtype).
AdamW moments updated outside no_grad	With create_graph gradients, exp_avg joined the autograd graph (memory leak; the spec says no graph).	@torch.no_grad() on step(); the closure still runs under enable_grad.
AdamW updated group 0, then rejected group 1	Invalid values must be rejected before any update.	Validate all groups first, then update.
Schedule accepted True and float s_w/s_c , rejected np.int64	Non-integer step indices must be rejected (bool is an int subclass).	numbers.Integral check excluding bool; return float.
Clipping norm overflowed in fp16	$300^2 > 65,504 \rightarrow \text{inf norm} \rightarrow \text{scale 0}$, which zeroes all gradients.	Square in $\geq \text{fp32}$.
make_submission.sh refused to package	Commented-out “NotImplementedError / TODO” lines in adapters.py matched its guard grep.	Deleted the leftover comment lines.
Report: “same seed for model init” claim; missing figure link	train.py never seeds the model; the image file did not exist.	Corrected the claim; removed the broken link.

None of these fixes changes fp32 numerics, so the trained model, the evaluation and the samples stayed valid, and no retraining was needed.

Part C — Formula and numbers cheat sheet

Item	Formula / value
Linear	$y = x W^T$, $W \in \mathbb{R}^{\text{out} \times \text{in}}$; $\sigma^2 = 2/(\text{in} + \text{out})$, truncated at $\pm 3\sigma$
Embedding init	$N(0, 1)$ truncated to $[-3, 3]$; RMSNorm $\gamma = 1$
RMSNorm	$\gamma \odot x / \sqrt{(\text{mean}(x^2) + \epsilon)}$, $\epsilon = 1\text{e-}5$, computed in $\geq \text{fp32}$
SiLU / SwiGLU	$x \sigma(x)$; $W_{\text{down}}(\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x)$, $d_{\text{ff}} \approx 8/3 d \rightarrow 1344$
RoPE	$\omega_k = \Theta^{-(2k-2)/d_h}$, $\phi = i\omega_k$; pairs (1,2),(3,4)...; tables (256, 16); Q and K only
Softmax	$\exp(z - \max) / \sum \exp(z - \max)$
Attention	$\text{softmax}(QK^T/\sqrt{d_k} + \text{mask}) V$; mask True = allowed; causal $j \leq i$
GQA	$g = h_q/h_{kv} = 4$; head $a = (h-1)g + r$; Q $[B, h_{kv}, g, n, d_h]$, K/V $[B, h_{kv}, n, d_h]$
Parameters	$2 V d + L(2d^2 + 2d h_{kv} d_h + 3d d_{\text{ff}} + 2d) + d = 19,272,192$
Cross-entropy	$-z_y + \text{logsumexp}(z)$, mean over positions; initial $\approx \ln 8192 = 9.01$
Perplexity	$\exp(\text{mean CE})$; yours $\exp(1.6114\dots) = 5.0100$
AdamW	$m = \beta_1 m + (1-\beta_1)g$; $v = \beta_2 v + (1-\beta_2)g^2$; $\hat{m}$, $\hat{v} \div (1-\beta^t)$; $\theta \leftarrow (1-\alpha\lambda)\theta - \alpha \hat{m}/(\sqrt{\hat{v}} + \epsilon)$
Schedule	warmup $(s/s_w)\alpha_{\text{max}}$; cosine $\alpha_{\text{min}} + \frac{1}{2}(1 + \cos \pi \cdot \text{progress})(\alpha_{\text{max}} - \alpha_{\text{min}})$; floor α_{min} after s_c
Clipping	if $\ g\ > M$: $g \leftarrow g \cdot M/(\ g\ + 1\text{e-}6)$; return the pre-clip norm
Batch sampling	$o \sim \text{randint}(0, N - n)$; $x = u[o:o+n]$, $y = u[o+1:o+n+1]$
Tokens	$B \cdot n \cdot n_{\text{acc}} = 65,536$ per update; $\times 10,000 = 655,360,000$; ≈ 34.0 tokens/parameter; 1.40 corpus-equivalents
Chinchilla	≈ 20 tokens/parameter $\rightarrow 385.4$ M tokens; corpus 466.9 M = 24.2 tokens/parameter
Memory	fp32 params 77.1 MB; + Adam $m, v \rightarrow$ about 231 MB checkpoint; fp16 export 38.5 MB
Standardized eval	seed 42, $100 \times 16 \times 256 = 409,600$ tokens; CE 1.6114, PPL 5.0100
Decoding	$\tau > 0$; $0 < p \leq 1$; keep the shortest sorted prefix with cumsum $\geq p$; stop after EOT

Part D — Practice quiz (answer before looking at Part E)

Mix of conceptual, calculation, and code-reading questions, in the style an instructor would set for this assignment. Aim to answer each in 1–3 sentences or with a short calculation.

§1 Overview and rules

- Q1. Which parts of `torch.nn` / `torch.optim` are you allowed to use?
- Q2. May you call `torch.nn.functional.softmax` inside your attention? Why or why not?
- Q3. Are `torch.sigmoid` (in `SiLU`) and `torch.logsumexp` (in cross-entropy) allowed?
- Q4. What is `tests/adapters.py` for, and what must it not contain?
- Q5. How is PA1 graded?
- Q6. Why can passing all public tests still lose marks?

§2 Environment, data, tokenizer

- Q7. What kind of tokenizer is used, and what is its vocabulary size?
- Q8. Why does byte-level BPE have no unknown-token problem?
- Q9. What does `<|endoftext|>` do in the training stream?
- Q10. Are the embedding table and LM head pretrained?
- Q11. Why should you not decode and re-encode the training stream?
- Q12. How many tokens are in the training and validation streams?
- Q13. What does ``uv run pytest -k "attention and not gqa"'` do, and what must you run before submitting?

§3 Architecture

- Q14. State the fixed values of `vocab_size`, `context_length`, `d_model`, `num_layers`, `n_q_heads`, `n_kv_heads`, `head_dim`, `d_ff`, `rope_theta` and `norm_eps`.
- Q15. A Linear layer has `in_features` = 512 and `out_features` = 1344. What is the weight shape, and how is it applied to `x` of shape $(B, n, 512)$?
- Q16. Why must `W` be stored as `(out, in)` and not transposed?
- Q17. Compute the Linear initialization standard deviation and truncation bound for $512 \rightarrow 1344$.
- Q18. How are the Embedding and RMSNorm parameters initialized?
- Q19. Write the RMSNorm formula. Which dimension is normalized?
- Q20. Give two differences between RMSNorm and LayerNorm, and a reason to prefer RMSNorm.
- Q21. Why does RMSNorm upcast `fp16/bf16` input to `fp32`, and what dtype does it return?
- Q22. Explain pre-norm vs post-norm and why this model uses pre-norm.
- Q23. Why does each block have two RMSNorm modules instead of sharing one?
- Q24. Write `SiLU` and state `SiLU(0)`. How does it differ from `ReLU`?
- Q25. Write the `SwiGLU` equation and give the shapes of its three matrices.
- Q26. Why is `d_ff` = 1344 rather than $4 \cdot 512 = 2048$?
- Q27. How many parameters does one `SwiGLU` FFN have?
- Q28. Which tensors does `RoPE` rotate, and why not the others?
- Q29. Give the `RoPE` frequency formula, and compute ω_1 and ω_{16} for $d_h = 32$, $\Theta = 10,000$.
- Q30. Write the adjacent-pair rotation for pair `k` at position `i`.
- Q31. Show why `RoPE` encodes relative position.
- Q32. What is the difference between the adjacent-pair and half-split `RoPE` conventions? Can you mix them?
- Q33. What shape are the `RoPE` cos/sin tables, and why are they non-persistent buffers?
- Q34. Why not implement `RoPE` with a dense $d_h \times d_h$ rotation matrix?

- Q35.** List the four input conditions RoPE must reject.
- Q36.** Compute $\text{softmax}([1000, 1001, 1002])$ and explain how you avoid overflow.
- Q37.** Why are attention scores divided by $\sqrt{d_k}$?
- Q38.** In this assignment, what does `mask = True` mean, and how is the mask applied?
- Q39.** Why must scaled dot-product attention raise an error when a query row is entirely masked?
- Q40.** Q has shape (2, 3, 4, 6), K (2, 3, 5, 6), V (2, 3, 5, 7). What is the output shape?
- Q41.** Which query-key pairs does the causal mask allow, and how can you build it?
- Q42.** With $h_q = 16$ and $h_{kv} = 4$, which K/V head does query head $a = 6$ (1-indexed) use, and what is r ?
- Q43.** Give the shapes of grouped Q , K/V and the score tensor in GQA.
- Q44.** Why must you not use `repeat_interleave` on K and V in GQA? What do you do instead?
- Q45.** Express MHA and MQA as special cases of GQA.
- Q46.** What are the shapes of W_Q , W_K , W_V and W_O , and how many attention parameters does one layer have?
- Q47.** What is the main practical advantage of GQA over MHA at inference time?
- Q48.** What goes wrong if you merge the attention heads in the wrong (h, r) order?
- Q49.** Why does the model reject sequences longer than `context_length`?
- Q50.** Derive the 19,272,192 parameter count.
- Q51.** Why does the model not add absolute position embeddings, and why is there no `softmax` in `forward()`?
- Q52.** What does “untied” mean for the embedding and LM head?

§4 Loss and optimization

- Q53.** Write numerically stable cross-entropy for logits z and target y .
- Q54.** Why not compute cross-entropy as $-\log(\text{softmax}(z))[y]$?
- Q55.** What loss should a freshly initialized model have on this vocabulary, and why?
- Q56.** Define perplexity and compute it for a mean CE of 1.6114.
- Q57.** Why exponentiate the mean CE instead of averaging per-batch perplexities?
- Q58.** Write the AdamW update, labelling bias correction and decoupled weight decay.
- Q59.** What is the size of the very first AdamW update, ignoring ϵ and weight decay?
- Q60.** Why is bias correction needed?
- Q61.** What is the difference between Adam + L2 and AdamW?
- Q62.** Why does AdamW keep a per-parameter step count t_p instead of using the global step s ?
- Q63.** What must AdamW do with a parameter whose `.grad` is `None`? With a sparse gradient?
- Q64.** What is lazy optimizer state, and why use it?
- Q65.** Why does `step()` run under `torch.no_grad()` while the closure runs under `torch.enable_grad()`?
- Q66.** Why validate each group’s `lr` / `betas` / `eps` / `weight_decay` inside `step()`, not just in `__init__`?
- Q67.** Your full checkpoints are about 231 MB. Explain that size.
- Q68.** Write the three branches of the warmup + cosine schedule.
- Q69.** With $\alpha_{\max} = 5e-4$, $\alpha_{\min} = 3e-5$, $s_w = 100$, $s_c = 9999$: find α_0 , α_{50} , α_{100} , α at $s \approx 5050$, and α_{9999} .
- Q70.** What does the schedule return at step 0 when `warmup = 0`?
- Q71.** Why use warmup at all?
- Q72.** Why hold the LR at $\alpha_{\min}$ after s_c instead of continuing the cosine formula?
- Q73.** Gradients are $[3, 0, 0]$ and $[0, 4, 0, 0]$ with max norm 1. What does clipping return, and what are the new gradients?
- Q74.** Why clip with one global factor instead of clipping each parameter separately?
- Q75.** When exactly is gradient clipping applied in a training step, and why return the pre-clip norm?

§5 Data, checkpoints, training loop

- Q76. Why load the token files with `np.memmap` instead of `np.fromfile` or `np.asarray(..., int64)`?
- Q77. Why is the dtype `<u2`, and why reject files with an odd byte length?
- Q78. For a stream of N tokens and sequence length n , what are the valid starts, and what `randint` call do you use?
- Q79. $N = 20$, $n = 5$: what is the largest possible start, and which tokens are x and y for start o ?
- Q80. What is wrong with `torch.randint(0, N - n + 1)`?
- Q81. Why does `get_batch` take a `torch.Generator` instead of relying on `torch.manual_seed`?
- Q82. Why are there two independent generators (train and validation)?
- Q83. What goes into a checkpoint, and what is `next_step` after completing step 499?
- Q84. Why checkpoint only between complete global steps?
- Q85. Define a global step. With $B = 16$, $n = 256$, $n_{\text{acc}} = 16$, how many tokens does one step use?
- Q86. Why divide each microbatch loss by n_{acc} before `backward()`?
- Q87. Is the logged training loss the scaled or the unscaled loss?
- Q88. Write the canonical per-step order.
- Q89. Why must validation come before checkpointing at the same step?
- Q90. Distinguish `model.eval()`, `torch.no_grad()` and `torch.inference_mode()`.
- Q91. Why does `evaluate_validation` save and restore the model's mode?
- Q92. What is the "overfit one fixed minibatch" check, and what result is expected?
- Q93. Your first learning rate logged at step 1 was 0. Is that a bug?

§6 Decoding

- Q94. Why must the prompt be non-empty?
- Q95. For logits $[2, 1, 0]$, give the probabilities at $\tau = 1$ and $\tau = 0.5$.
- Q96. What happens as $\tau \rightarrow 0^+$, and as τ grows large?
- Q97. Probabilities are $[0.5, 0.3, 0.15, 0.05]$. Which tokens are kept, and what are the renormalized probabilities, for $p = 0.8$ and $p = 0.9$?
- Q98. Why is top-p called adaptive, compared with top-k or temperature?
- Q99. After sampling from the sorted, filtered distribution, how do you get the token ID?
- Q100. Why crop the sequence to the last `context_length` tokens at every step?
- Q101. Why must the `context_length` argument equal `model.context_length`?
- Q102. What happens when the EOT token is generated?
- Q103. What is the cost of decoding without a KV cache?
- Q104. Why is $\tau = 0$ rejected rather than treated as greedy?
- Q105. Which state must `generate()` save and restore, and under what contexts does it run?

§7 Training, evaluation, your results

- Q106. Why does a short tuning run keep $s_c = 9999$ instead of compressing the cosine to 1,500 steps?
- Q107. List the controls needed for a fair tuning comparison.
- Q108. Which control did your tuning runs miss, and how did you account for it?
- Q109. Why did changing α_{min} barely affect the 1,500-step runs?
- Q110. Why is the effective-batch comparison confounded?
- Q111. Which three hyperparameters did you change for the final run, and how strong was the evidence for each?
- Q112. How many tokens did the final run sample, and how does that relate to the Chinchilla heuristic?

- Q113.** Describe the standardized evaluation exactly.
- Q114.** What were your standardized CE and PPL?
- Q115.** Why use a fresh seed-42 generator for the final evaluation instead of the training-time validation loss?
- Q116.** Your last training-time validation was 1.6437 but the standardized one was 1.6114. Is that a contradiction?
- Q117.** Why is a higher distinct-2 at $T = 1.3$ not evidence of better text?
- Q118.** Summarize the temperature and top-p trends you measured.
- Q119.** Which decoding setting did you consider the best balance, and why?
- Q120.** What exactly must `final_model.pt` contain?
- Q121.** Why fp16, and how large is the file?
- Q122.** What does the submission zip contain, and what must you do before uploading?

Debugging and code-reading

- Q123.** A RoPE implementation returns fp32 when given bf16 input. Why, and how do you fix it?
- Q124.** An AdamW step updates `exp_avg` without `torch.no_grad()`. When does this break, and what is the symptom?
- Q125.** A training loop calls `loss.backward()` for each of 16 microbatches without dividing by 16. What changes?
- Q126.** A gradient-clipping function squares fp16 gradients of about 300. What happens?
- Q127.** A student reports perplexity as the mean of 100 per-batch perplexities. Is that correct?
- Q128.** After attention, a student reshapes $[B, h_{kv}, g, n, d_h]$ by transposing to $[B, n, g, h_{kv}, d_h]$ before flattening. What is wrong?
- Q129.** A schedule function checks `isinstance(step, int)`. Which invalid input slips through?
- Q130.** A student registers the RoPE cos/sin tables with `persistent=True`. What breaks?
- Q131.** Validation code calls `model.eval()` but never restores the mode. Why is that a problem even without dropout?
- Q132.** Why is ``mask.sum(-1) == 0 → raise`` the right check before masked softmax?
- Q133.** `make_submission.sh` prints “tests/adapters.py still contains an unfinished adapter” even though all tests pass. What is the cause?

Part E — Answer key

§1 Overview and rules

Q1. Which parts of `torch.nn` / `torch.optim` are you allowed to use?

Answer: `nn.Parameter`, container classes (`nn.Module`, `ModuleList`, `Sequential`, ...), `torch.nn.init` utilities (including `trunc_normal_`), and the `torch.optim.Optimizer` base class. Nothing else from `torch.nn`, `torch.nn.functional` or `torch.optim`.

Q2. May you call `torch.nn.functional.softmax` inside your attention? Why or why not?

Answer: No, for two reasons: `torch.nn.functional` is forbidden, and `softmax` is itself a deliverable, so you may not call a built-in equivalent.

Q3. Are `torch.sigmoid` (in `SiLU`) and `torch.logsumexp` (in cross-entropy) allowed?

Answer: Yes. Both are explicitly permitted by the manual and live in the top-level `torch` namespace.

Q4. What is `tests/adapters.py` for, and what must it not contain?

Answer: It is glue between the tests and your code: import your module, construct it, load the supplied weights, call it, and return the result. It must not contain assignment maths (no matrix multiplications, attention or optimization logic).

Q5. How is PA1 graded?

Answer: 70% unit tests (public plus hidden tests through the same adapters) and 30% `REPORT.md`.

Q6. Why can passing all public tests still lose marks?

Answer: Hidden tests check other parts of the written contract. Example: your RoPE returned `fp32` for `bf16` input and still passed the public tests, but the manual requires the same dtype as the input.

§2 Environment, data, tokenizer

Q7. What kind of tokenizer is used, and what is its vocabulary size?

Answer: A fixed byte-level BPE tokenizer with 8,192 tokens.

Q8. Why does byte-level BPE have no unknown-token problem?

Answer: It works over UTF-8 bytes. Every one of the 256 byte values is in the base vocabulary, so any text can be encoded; merges only shorten common sequences.

Q9. What does `<|endoftext|>` do in the training stream?

Answer: It separates consecutive stories (`encode(story1)`, `EOT`, `encode(story2)`, `EOT`, ...), so a sampled window can cross a document boundary and the model learns where stories end. In generation, producing `EOT` ends the story.

Q10. Are the embedding table and LM head pretrained?

Answer: No. They are ordinary randomly initialized parameters learned during training; only the tokenizer is fixed.

Q11. Why should you not decode and re-encode the training stream?

Answer: It is already token IDs. Re-tokenizing would waste time and could produce different tokenization; use the IDs directly.

Q12. How many tokens are in the training and validation streams?

Answer: Train 466,876,982 tokens (2,119,719 stories); validation 4,692,376 tokens (21,990 stories).

Q13. What does ``uv run pytest -k "attention and not gqa"'` do, and what must you run before submitting?

Answer: `-k` selects tests whose collected names match the expression (here names containing “attention” but not “gqa”). Before submitting, run the full suite: ``uv run pytest`` with no path or `-k` filter.

§3 Architecture

Q14. State the fixed values of vocab_size, context_length, d_model, num_layers, n_q_heads, n_kv_heads, head_dim, d_ff, rope_theta and norm_eps.

Answer: 8192, 256, 512, 4, 16, 4, 32, 1344, 10000, 1e-5 (group size $g = 4$).

Q15. A Linear layer has in_features = 512 and out_features = 1344. What is the weight shape, and how is it applied to x of shape $(B, n, 512)$?

Answer: W has shape $(1344, 512)$, i.e. (out, in) . Output = $x @ W.T$, with shape $(B, n, 1344)$.

Q16. Why must W be stored as (out, in) and not transposed?

Answer: It is PyTorch's `nn.Linear` convention, and the tests inject weights in exactly that orientation. A transposed store gives wrong outputs (or shape errors) in every weight-loading test.

Q17. Compute the Linear initialization standard deviation and truncation bound for $512 \rightarrow 1344$.

Answer: $\sigma = \sqrt{2/(512+1344)} = \sqrt{2/1856} \approx 0.0328$; truncation at $\pm 3\sigma \approx \pm 0.098$.

Q18. How are the Embedding and RMSNorm parameters initialized?

Answer: Embedding: $N(0, 1)$ truncated to $[-3, 3]$. RMSNorm gain: all ones.

Q19. Write the RMSNorm formula. Which dimension is normalized?

Answer: $\text{RMSNorm}(x) = y \odot x / \sqrt{(\text{mean}(x^2) + \epsilon)}$, over the last (`d_model`) dimension only; no mean subtraction and no bias.

Q20. Give two differences between RMSNorm and LayerNorm, and a reason to prefer RMSNorm.

Answer: RMSNorm has no centering (mean subtraction) and no bias; it only rescales by the root mean square. It is cheaper, and in practice (e.g. LLaMA) it works as well, because the re-scaling is what stabilizes training.

Q21. Why does RMSNorm upcast fp16/bf16 input to fp32, and what dtype does it return?

Answer: Squares overflow in fp16 (max 65,504; $300^2 = 90,000 \rightarrow \text{inf}$) and the mean of squares loses precision. It computes in fp32 and returns the original input dtype.

Q22. Explain pre-norm vs post-norm and why this model uses pre-norm.

Answer: Post-norm: $x = \text{Norm}(x + f(x))$. Pre-norm: $x = x + f(\text{Norm}(x))$, plus a final norm after the last block. Pre-norm keeps an un-normalized residual path from the input embedding to the output, which gives better gradient flow and more stable training.

Q23. Why does each block have two RMSNorm modules instead of sharing one?

Answer: The attention and FFN sublayers need independently learned gains; the spec requires distinct modules with independent parameters.

Q24. Write SiLU and state $\text{SiLU}(0)$. How does it differ from ReLU?

Answer: $\text{SiLU}(x) = x \cdot \sigma(x) = x / (1 + e^{-x})$; $\text{SiLU}(0) = 0$. It is smooth at 0 and allows small negative outputs, whereas ReLU has a kink at 0 and outputs 0 for all negative inputs.

Q25. Write the SwiGLU equation and give the shapes of its three matrices.

Answer: $W_{\text{down}} [\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x]$; $W_{\text{gate}}, W_{\text{up}} : (1344, 512)$; $W_{\text{down}} : (512, 1344)$. All bias-free.

Q26. Why is `d_ff` = 1344 rather than $4 \cdot 512 = 2048$?

Answer: SwiGLU has three matrices, so `d_ff` $\approx (8/3)d = 1365.3$ keeps the parameter count close to a classic two-matrix FFN of width $4d$. It is rounded to a multiple of 64 ($21 \times 64 = 1344$) for hardware efficiency.

Q27. How many parameters does one SwiGLU FFN have?

Answer: $3 \times 512 \times 1344 = 2,064,384$.

Q28. Which tensors does RoPE rotate, and why not the others?

Answer: Queries and keys only. Position must affect the query-key dot products (who attends to whom); values carry content and are left unchanged.

Q29. Give the RoPE frequency formula, and compute ω_1 and ω_{16} for $d_h = 32$, $\Theta = 10,000$.

Answer: $\omega_k = \Theta^{-(2k-2)/d_h}$. $\omega_1 = 10000^0 = 1$; $\omega_{16} = 10000^{-30/32} = 10^{-3.75} \approx 1.78 \times 10^{-4}$.

Q30. Write the adjacent-pair rotation for pair k at position i .

Answer: $x'_{2k-1} = x_{2k-1} \cos \varphi - x_{2k} \sin \varphi$; $x'_{2k} = x_{2k-1} \sin \varphi + x_{2k} \cos \varphi$, with $\varphi = i\omega_k$.

Q31. Show why RoPE encodes relative position.

Answer: Each 2-D block satisfies $R(\varphi_i)^T R(\varphi_j) = R(\varphi_j - \varphi_i) = R((j-i)\omega)$. So $(R_i q) \cdot (R_j k) = q^T R_{j-i} k$, which depends only on the offset $j - i$.

Q32. What is the difference between the adjacent-pair and half-split RoPE conventions? Can you mix them?

Answer: Adjacent pairs rotate $(x_1, x_2), (x_3, x_4), \dots$; half-split (GPT-NeoX) pairs x_k with $x_{k+d/2}$. Both are valid RoPE but they are different coordinate layouts; mixing them gives wrong results against the injected weights. The assignment uses adjacent pairs.

Q33. What shape are the RoPE cos/sin tables, and why are they non-persistent buffers?

Answer: $(\text{context_length}, \text{head_dim}/2) = (256, 16)$. As buffers they move with `.to(device)` and are not trained. Non-persistent means they are excluded from `state_dict`, because they can be recomputed exactly and should not inflate the saved model (which must hold exactly 19,272,192 values).

Q34. Why not implement RoPE with a dense $d_h \times d_h$ rotation matrix?

Answer: The matrix is block-diagonal and mostly zeros; multiplying by it costs $O(d_h^2)$ per vector instead of $O(d_h)$ element-wise work. The spec explicitly forbids it.

Q35. List the four input conditions RoPE must reject.

Answer: Odd `head_dim`, a mismatched final dimension, non-integer (e.g. float or bool) positions, and positions outside $[0, \text{context_length})$.

Q36. Compute $\text{softmax}([1000, 1001, 1002])$ and explain how you avoid overflow.

Answer: Subtract the max, giving $\text{softmax}([-2, -1, 0]) = \text{softmax}([0, 1, 2]) \approx [0.090, 0.245, 0.665]$. Softmax is invariant to adding a constant, and the shift keeps exp from overflowing ($\exp(1000) = \text{inf}$ would give $\text{inf}/\text{inf} = \text{NaN}$).

Q37. Why are attention scores divided by $\sqrt{d_k}$?

Answer: For unit-variance independent components, $q \cdot k$ has variance d_k . Large scores saturate softmax into near one-hot outputs with tiny gradients; dividing by $\sqrt{d_k}$ restores unit variance.

Q38. In this assignment, what does `mask = True` mean, and how is the mask applied?

Answer: True means query i may attend to key j . Where the mask is False, the score is set to $-\infty$ before softmax, so that probability becomes exactly 0.

Q39. Why must scaled dot-product attention raise an error when a query row is entirely masked?

Answer: The whole row would be $-\infty$, so max-shift and exp give NaN. The distribution is undefined, and the spec requires failing clearly rather than returning non-finite values.

Q40. Q has shape $(2, 3, 4, 6)$, K $(2, 3, 5, 6)$, V $(2, 3, 5, 7)$. What is the output shape?

Answer: $(2, 3, 4, 7)$: $(\dots, n_q, d_v)$. Here $n_q = 4 \neq n_{kv} = 5$ and $d_v = 7 \neq d_k = 6$, which is allowed.

Q41. Which query-key pairs does the causal mask allow, and how can you build it?

Answer: Key j is allowed for query i iff $j \leq i$. Build it with `torch.tril(torch.ones(n, n, dtype=bool))` or a broadcast comparison of index vectors.

Q42. With $h_q = 16$ and $h_{kv} = 4$, which K/V head does query head $a = 6$ (1-indexed) use, and what is r ?

Answer: $g = 4$ and $a = (h-1)g + r$ gives $6 = 4 + 2$, so $h = 2$ and $r = 2$. Query heads 5–8 share K/V head 2.

Q43. Give the shapes of grouped Q , K/V and the score tensor in GQA.

Answer: Q : $[B, h_{kv}, g, n, d_h]$; K, V : $[B, h_{kv}, n, d_h]$; S and A : $[B, h_{kv}, g, n, n]$.

Q44. Why must you not use `repeat_interleave` on K and V in GQA? What do you do instead?

Answer: Repeating would materialize g copies of K and V , wasting memory (the spec forbids it). Instead, keep Q grouped as $[B, h_{kv}, g, n, d_h]$ and use `einsum` or broadcasting, so each K_h, V_h is shared across its g query heads.

Q45. Express MHA and MQA as special cases of GQA.

Answer: MHA: $h_{kv} = h_q$ ($g = 1$). MQA: $h_{kv} = 1$ ($g = h_q$). GQA: $1 < h_{kv} < h_q$.

- Q46.** What are the shapes of W_Q , W_K , W_V and W_O , and how many attention parameters does one layer have?
Answer: W_Q 512×512, W_K 128×512, W_V 128×512, W_O 512×512; total 655,360 (compared with 1,048,576 for full MHA).
- Q47.** What is the main practical advantage of GQA over MHA at inference time?
Answer: The KV cache has 4 heads instead of 16, so it is 4× smaller. That means less memory and memory bandwidth per generated token, with quality close to MHA.
- Q48.** What goes wrong if you merge the attention heads in the wrong (h, r) order?
Answer: The output projection's columns expect head order $a = (h-1)g + r$. A wrong merge permutes heads, so the output is silently wrong and the weight-loading tests fail.
- Q49.** Why does the model reject sequences longer than context_length?
Answer: The RoPE tables only cover positions 0...255, and the model was only trained on windows of length ≤ 256 ; the spec requires $1 \leq n \leq \text{context_length}$.
- Q50.** Derive the 19,272,192 parameter count.
Answer: Embedding plus head: $2 \times 8192 \times 512 = 8,388,608$. Per layer: 524,288 (Q, O) + 131,072 (K, V) + 2,064,384 (FFN) + 1,024 (norms) = 2,720,768; $\times 4 = 10,883,072$. Plus the final norm 512. Total 19,272,192.
- Q51.** Why does the model not add absolute position embeddings, and why is there no softmax in forward()?
Answer: Position comes from RoPE inside attention. forward returns logits; softmax is applied only where needed (inside cross-entropy, or when sampling).
- Q52.** What does "untied" mean for the embedding and LM head?
Answer: They are two separate parameter matrices of shape (8192, 512), rather than sharing one matrix. This adds 4,194,304 parameters.

§4 Loss and optimization

- Q53.** Write numerically stable cross-entropy for logits z and target y.
Answer: $\ell = -z_y + \log \sum_c e^{z_c}$, computing the log-sum-exp with a max shift ($m + \log \sum e^{z_c - m}$), averaged over all leading positions.
- Q54.** Why not compute cross-entropy as $-\log(\text{softmax}(z))[y]$?
Answer: It is less stable (log of a possibly underflowed probability) and materializes a full probability tensor; the spec forbids torch.softmax / log_softmax here. Gather the target logit directly instead.
- Q55.** What loss should a freshly initialized model have on this vocabulary, and why?
Answer: About $\ln 8192 \approx 9.01$ nats (PPL ≈ 8192), because initial predictions are close to uniform. Your first logged step was 9.01.
- Q56.** Define perplexity and compute it for a mean CE of 1.6114.
Answer: PPL = $\exp(\text{mean CE}) = \exp(1.6114...) \approx 5.01$, i.e. about as uncertain as a uniform choice among 5 tokens.
- Q57.** Why exponentiate the mean CE instead of averaging per-batch perplexities?
Answer: Perplexity is defined from the average per-token log-loss. By Jensen's inequality, $\text{mean}(\exp) \geq \exp(\text{mean})$, so averaging per-batch PPLs overstates perplexity.
- Q58.** Write the AdamW update, labelling bias correction and decoupled weight decay.
Answer: $m = \beta_1 m + (1-\beta_1)g$; $v = \beta_2 v + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$ (bias correction); $\theta \leftarrow (1-\alpha)\theta$ (decoupled decay); $\theta \leftarrow \theta - \alpha \hat{m}/(\sqrt{\hat{v}} + \epsilon)$.
- Q59.** What is the size of the very first AdamW update, ignoring ϵ and weight decay?
Answer: At $t = 1$, $\hat{m} = g$ and $\hat{v} = g^2$, so the update is $-\alpha \cdot g/|g| = -\alpha \cdot \text{sign}(g)$: every parameter moves by about α .
- Q60.** Why is bias correction needed?
Answer: m and v are initialized to 0, so their early EMAs are biased toward 0. Dividing by $1-\beta^t$ removes this bias; the correction fades as t grows.

Q61. What is the difference between Adam + L2 and AdamW?

Answer: Adam + L2 adds $\lambda\theta$ to the gradient, so it is then divided by $\sqrt{\hat{v}}$ and the effective decay differs per parameter. AdamW applies $\theta \leftarrow (1-\alpha\lambda)\theta$ directly, decoupled from the adaptive scaling.

Q62. Why does AdamW keep a per-parameter step count t_p instead of using the global step s ?

Answer: Some parameters can be skipped (grad None) on some steps. Bias correction must count only the updates that parameter actually received.

Q63. What must AdamW do with a parameter whose .grad is None? With a sparse gradient?

Answer: Skip it completely: no weight decay, no state creation, no step increment. Reject sparse gradients with an error.

Q64. What is lazy optimizer state, and why use it?

Answer: state[p] (step = 0, exp_avg, exp_avg_sq as zeros_like(p)) is created only on the first update where p has a gradient. This saves memory for parameters that never receive gradients.

Q65. Why does step() run under torch.no_grad() while the closure runs under torch.enable_grad()?

Answer: Updates must not build an autograd graph; otherwise graph-carrying gradients make the moments part of the graph, leaking memory. The closure must compute the loss and gradients even if the caller disabled grad.

Q66. Why validate each group's lr / betas / eps / weight_decay inside step(), not just in __init__?

Answer: Callers change group values after construction (the scheduler writes group["lr"] every step). Invalid values must be rejected before any parameter is modified.

Q67. Your full checkpoints are about 231 MB. Explain that size.

Answer: fp32 parameters: $19,272,192 \times 4 \text{ B} \approx 77.1 \text{ MB}$. AdamW adds exp_avg and exp_avg_sq of the same size, so $3 \times 77.1 \approx 231 \text{ MB}$.

Q68. Write the three branches of the warmup + cosine schedule.

Answer: $s < s_w: (s/s_w)\alpha_{\max}$. $s_w \leq s \leq s_c: \alpha_{\min} + \frac{1}{2}(1+\cos(\pi(s-s_w)/(s_c-s_w)))(\alpha_{\max}-\alpha_{\min})$. $s > s_c: \alpha_{\min}$.

Q69. With $\alpha_{\max} = 5e-4$, $\alpha_{\min} = 3e-5$, $s_w = 100$, $s_c = 9999$: find α_0 , α_{50} , α_{100} , α at $s \approx 5050$, and α_{9999} .

Answer: $\alpha_0 = 0$; $\alpha_{50} = 2.5e-4$; $\alpha_{100} = 5e-4$ (peak); at the cosine midpoint ($s \approx 5050$), $\alpha \approx 3e-5 + 0.5 \times 4.7e-4 \approx 2.65e-4$; $\alpha_{9999} = 3e-5$.

Q70. What does the schedule return at step 0 when warmup = 0?

Answer: $\alpha_{\max}$: step 0 is already in the cosine branch at progress 0.

Q71. Why use warmup at all?

Answer: Early on, the weights are random and Adam's second-moment estimates come from very few samples, so full-size steps can be erratic or diverge. Ramping the LR up stabilizes the start.

Q72. Why hold the LR at $\alpha_{\min}$ after s_c instead of continuing the cosine formula?

Answer: Cosine is periodic; past s_c the formula would increase the learning rate again.

Q73. Gradients are [3, 0, 0] and [0, 4, 0] with max norm 1. What does clipping return, and what are the new gradients?

Answer: Pre-clip norm = $\sqrt{9+16} = 5.0$ is returned. Scale = $1/(5 + 1e-6) \approx 0.2$, giving [0.6, 0, 0] and [0, 0.8, 0, 0] (norm about 1).

Q74. Why clip with one global factor instead of clipping each parameter separately?

Answer: A single global factor preserves the direction of the full gradient vector and only limits its length; per-parameter clipping would distort the direction.

Q75. When exactly is gradient clipping applied in a training step, and why return the pre-clip norm?

Answer: Once per global step, after all accumulation backward passes and before optimizer.step(). The pre-clip norm is logged to monitor stability (spikes signal trouble).

§5 Data, checkpoints, training loop

Q76. Why load the token files with `np.memmap` instead of `np.fromfile` or `np.asarray(..., int64)`?

Answer: `memmap` maps the file lazily, so only the sampled pages are read. An `int64` copy of 466.9 M tokens would take about 3.7 GB of RAM, versus a 934 MB file.

Q77. Why is the dtype `<u2`, and why reject files with an odd byte length?

Answer: IDs are below $8192 < 65,536$, so each fits in 2 bytes, stored little-endian (`<`). An odd byte count cannot be a whole number of 2-byte IDs, so the file is corrupt or of the wrong type.

Q78. For a stream of N tokens and sequence length n , what are the valid starts, and what `randint` call do you use?

Answer: $0 \leq o \leq N - n - 1$, so `torch.randint(0, N - n, (B,), generator=g)`. The upper bound is exclusive, and the last window reads `u[N-n-1 : N]`.

Q79. $N = 20$, $n = 5$: what is the largest possible start, and which tokens are x and y for start o ?

Answer: The largest start is 14. $x = u[o : o+5]$, $y = u[o+1 : o+6]$ (6 tokens are read in total).

Q80. What is wrong with `torch.randint(0, N - n + 1)`?

Answer: It can sample $o = N - n$, whose target window needs `u[N]`, one position past the end of the stream.

Q81. Why does `get_batch` take a `torch.Generator` instead of relying on `torch.manual_seed`?

Answer: An explicit generator gives a reproducible batch stream without resetting global RNG state, and lets training and validation have independent streams.

Q82. Why are there two independent generators (train and validation)?

Answer: So that validating more or less often does not change the sequence of training batches.

Q83. What goes into a checkpoint, and what is `next_step` after completing step 499?

Answer: Model state_dict, optimizer state_dict, `next_step`, and both generator states. `next_step = 500`.

Q84. Why checkpoint only between complete global steps?

Answer: Accumulated gradients and a partially advanced training generator mid-step are not saved, so resuming from mid-step would not reproduce the trajectory.

Q85. Define a global step. With $B = 16$, $n = 256$, $n_{\text{acc}} = 16$, how many tokens does one step use?

Answer: One optimizer update. $16 \times 256 \times 16 = 65,536$ tokens (effective batch 256 sequences).

Q86. Why divide each microbatch loss by n_{acc} before `backward()`?

Answer: So the accumulated gradient equals the gradient of the mean loss over the n_{acc} minibatches. Without it, the gradient is n_{acc} × too large, which changes the effective LR and triggers clipping differently.

Q87. Is the logged training loss the scaled or the unscaled loss?

Answer: The unscaled mean of the microbatch cross-entropies (the division is only for backward).

Q88. Write the canonical per-step order.

Answer: train mode → α_s → `zero_grad` → n_{acc} minibatches with backward accumulation → clip → `optimizer.step` → validate → log → checkpoint. The final step forces all three.

Q89. Why must validation come before checkpointing at the same step?

Answer: The checkpoint then stores the validation generator state after the validation batches drawn at that boundary, so a resumed run continues the same validation stream exactly.

Q90. Distinguish `model.eval()`, `torch.no_grad()` and `torch.inference_mode()`.

Answer: `eval()` switches module behaviour (e.g. dropout) but does not disable autograd. `no_grad()` stops gradient recording. `inference_mode()` does that and also skips extra autograd bookkeeping, so it is faster and lighter. Validation uses `eval()` + `inference_mode()`.

Q91. Why does `evaluate_validation` save and restore the model's mode?

Answer: It must not change the caller's state: it records `was_training`, calls `eval()`, and at the end calls `model.train(was_training)`.

Q92. What is the “overfit one fixed minibatch” check, and what result is expected?

Answer: Repeatedly train on the same batch with a small model; a correct pipeline drives the loss to near 0. Yours went from 9.03 to 0.004 in 300 steps.

Q93. Your first learning rate logged at step 1 was 0. Is that a bug?

Answer: No. With warmup, the formula gives $\alpha_0 = (0/s_w) \cdot \alpha_{\max} = 0$, so the first update uses LR 0 by definition.

§6 Decoding

Q94. Why must the prompt be non-empty?

Answer: The first next-token distribution comes from the logits at the final prompt position; an empty prompt has no final position.

Q95. For logits [2, 1, 0], give the probabilities at $\tau = 1$ and $\tau = 0.5$.

Answer: $\tau = 1$: $\approx [0.665, 0.245, 0.090]$. $\tau = 0.5$ (logits become [4, 2, 0]): $\approx [0.867, 0.117, 0.016]$, which is sharper.

Q96. What happens as $\tau \rightarrow 0^+$, and as τ grows large?

Answer: $\tau \rightarrow 0^+$ approaches argmax (greedy, deterministic, repetitive). Large τ approaches a uniform distribution (random, incoherent).

Q97. Probabilities are [0.5, 0.3, 0.15, 0.05]. Which tokens are kept, and what are the renormalized probabilities, for $p = 0.8$ and $p = 0.9$?

Answer: $p = 0.8$: cumulative sums [0.5, 0.8, ...], so keep 2 tokens: [0.625, 0.375]. $p = 0.9$: the first cumulative sum ≥ 0.9 is 0.95, so keep 3 tokens: [0.526, 0.316, 0.158].

Q98. Why is top-p called adaptive, compared with top-k or temperature?

Answer: The size of the kept set depends on the model’s confidence: a few tokens when it is sure, many when it is unsure. Temperature changes every step’s distribution, and top-k always keeps k tokens.

Q99. After sampling from the sorted, filtered distribution, how do you get the token ID?

Answer: `torch.multinomial` returns a rank in sorted order; map it back through the saved sorted indices (`sorted_indices[rank]`). Equivalently, scatter the filtered probabilities back to vocabulary order before sampling.

Q100. Why crop the sequence to the last `context_length` tokens at every step?

Answer: The model rejects sequences longer than 256 (RoPE tables and training length), but the full output may grow beyond that; only the most recent 256 tokens are fed in.

Q101. Why must the `context_length` argument equal `model.context_length`?

Answer: So generation cannot silently crop to a different window than the model supports; a mismatch is rejected.

Q102. What happens when the EOT token is generated?

Answer: Generation stops immediately after it, and the EOT token is kept in the returned sequence.

Q103. What is the cost of decoding without a KV cache?

Answer: Each new token re-runs the full cropped context (up to 256 tokens) through the model; past keys and values are recomputed instead of reused.

Q104. Why is $\tau = 0$ rejected rather than treated as greedy?

Answer: Dividing the logits by 0 is undefined; the spec requires $\tau > 0$ (greedy decoding would be a separate argmax path).

Q105. Which state must `generate()` save and restore, and under what contexts does it run?

Answer: The model’s training/eval mode. It runs the loop under `model.eval()` and `torch.inference_mode()`, then calls `model.train(was_training)`.

§7 Training, evaluation, your results

Q106. Why does a short tuning run keep $s_c = 9999$ instead of compressing the cosine to 1,500 steps?

Answer: It must see the same learning-rate prefix the full 10,000-step run will see. A compressed schedule would test a different (much faster-decaying) LR curve.

Q107. List the controls needed for a fair tuning comparison.

Answer: Same update cap, model-initialization seed, training and validation generator seeds, evaluation frequency and validation procedure; change only one hyperparameter at a time.

Q108. Which control did your tuning runs miss, and how did you account for it?

Answer: The model initialization seed: train.py never calls torch.manual_seed, so each run started from different random weights. You used the $\alpha_{\min}$ runs (LR almost identical to baseline, loss differs by about 0.013) as a noise probe, and treated differences of about 0.02 or less as noise.

Q109. Why did changing $\alpha_{\min}$ barely affect the 1,500-step runs?

Answer: With $s_c = 9999$, the LR at step 1,500 has only decayed from $3e-4$ to about $2.88e-4$, so $\alpha_{\min}$ barely enters the formula yet.

Q110. Why is the effective-batch comparison confounded?

Answer: With a fixed step cap, batch 64 and 128 see $4\times$ and $2\times$ fewer tokens (24.6 M, 49.2 M vs 98.3 M), so you cannot separate the batch-size effect from the data-quantity effect.

Q111. Which three hyperparameters did you change for the final run, and how strong was the evidence for each?

Answer: $\alpha_{\max}$ $3e-4 \rightarrow 5e-4$ (strong: -0.122); warmup 200 $\rightarrow$ 100 (weak: -0.014 , within noise); β_2 $0.95 \rightarrow 0.999$ (weak: -0.014 , within noise).

Q112. How many tokens did the final run sample, and how does that relate to the Chinchilla heuristic?

Answer: $10,000 \times 65,536 = 655,360,000 \approx 34$ tokens/parameter, against the ~ 20 tokens/parameter heuristic (≈ 385 M). That is about 1.40 corpus-equivalents, sampled with replacement, so not literal epochs.

Q113. Describe the standardized evaluation exactly.

Answer: A fresh torch.Generator seeded with 42; 100 validation batches of 16×256 (409,600 tokens); the model in eval mode without gradients; mean of the 100 batch CEs, then one exponentiation for PPL.

Q114. What were your standardized CE and PPL?

Answer: CE = 1.6114 nats/token, PPL = 5.0100. The same values were reproduced on the submitted fp16 final_model.pt on CPU.

Q115. Why use a fresh seed-42 generator for the final evaluation instead of the training-time validation loss?

Answer: The training-time validation generator has advanced a different amount depending on how often you validated, and uses fewer batches. A fresh fixed seed makes the number reproducible and comparable across submissions.

Q116. Your last training-time validation was 1.6437 but the standardized one was 1.6114. Is that a contradiction?

Answer: No. They use different validation windows (seed 43, already advanced, vs a fresh seed 42) and different batch counts; the gap of about 0.03 is sampling variation. The standardized number is the more reliable one.

Q117. Why is a higher distinct-2 at $T = 1.3$ not evidence of better text?

Answer: The extra variety comes from errors, non-words and incoherent tokens. Variety metrics reward randomness as much as creativity, so coherence has to be judged separately (by reading).

Q118. Summarize the temperature and top-p trends you measured.

Answer: T $0.7 \rightarrow 1.0 \rightarrow 1.3$: distinct-2 $0.868 \rightarrow 0.918 \rightarrow 0.966$, repeated 4-grams $2.0\% \rightarrow 0.5\% \rightarrow 0.1\%$. p $1.0 \rightarrow 0.9 \rightarrow 0.8$: distinct-2 $0.918 \rightarrow 0.900 \rightarrow 0.891$, repeated 4-grams $0.5\% \rightarrow 1.1\% \rightarrow 1.4\%$.

Q119. Which decoding setting did you consider the best balance, and why?

Answer: $T = 1.0$, $p = 0.9$: variety (0.900) close to the untruncated setting, lower repetition than $T = 0.7$, and samples that were grammatical with only small logical slips. Coherence was judged manually, so this is an observation, not a general rule.

Q120. What exactly must `final_model.pt` contain?

Answer: A plain `state_dict` of the model only: every tensor on CPU, floating tensors cast to fp16 (non-floating dtypes preserved), exactly 19,272,192 values, and no optimizer state, RNG state or hyperparameters.

Q121. Why fp16, and how large is the file?

Answer: It halves the size relative to fp32 ($19,272,192 \times 2 \text{ B} \approx 38.5 \text{ MB}$ vs 77.1 MB) and is adequate for inference; the grader loads it via `get_transformer_lm`.

Q122. What does the submission zip contain, and what must you do before uploading?

Answer: `src/`, `tests/adapters.py`, `REPORT.md`, `report_assets/`, `final_model.pt` (no data, checkpoints or caches). Rename it to `<roll_number>_pa1.zip`.

Debugging and code-reading

Q123. A RoPE implementation returns fp32 when given bf16 input. Why, and how do you fix it?

Answer: Multiplying bf16 by fp32 cos/sin buffers promotes the result to fp32. Compute in `promote_types(x.dtype, fp32)` and cast the result back with `.to(x.dtype)`.

Q124. An AdamW step updates `exp_avg` without `torch.no_grad()`. When does this break, and what is the symptom?

Answer: When gradients carry a graph (`backward(create_graph=True)`), `exp_avg.add_(grad)` makes the moment buffers require grad and join the graph, so memory grows every step. Fix: decorate `step()` with `@torch.no_grad()`.

Q125. A training loop calls `loss.backward()` for each of 16 microbatches without dividing by 16. What changes?

Answer: Gradients are $16\times$ the mean-loss gradient: effectively a much larger step size, and the clipping threshold is hit far more often.

Q126. A gradient-clipping function squares fp16 gradients of about 300. What happens?

Answer: $300^2 = 90,000 > 65,504 \rightarrow \text{inf}$. The norm becomes inf, the scale $M/\text{inf} = 0$, and all gradients are zeroed. Fix: accumulate squares in fp32.

Q127. A student reports perplexity as the mean of 100 per-batch perplexities. Is that correct?

Answer: No. By Jensen's inequality it overstates perplexity; average the CE first, then exponentiate once.

Q128. After attention, a student reshapes $[B, h_{kv}, g, n, d_h]$ by transposing to $[B, n, g, h_{kv}, d_h]$ before flattening. What is wrong?

Answer: The flattened head order becomes r-major instead of $a = (h-1)g + r$, so heads are permuted relative to W_O 's columns and the output is wrong. Keep (h, r) order: reshape to $[B, h_q, n, d_h]$, transpose to $[B, n, h_q, d_h]$, then flatten.

Q129. A schedule function checks `isinstance(step, int)`. Which invalid input slips through?

Answer: `bool`: `True` and `False` are `int` subclasses, so `True` is accepted as step 1. Check `isinstance(step, numbers.Integral)` and not `isinstance(step, bool)`.

Q130. A student registers the RoPE cos/sin tables with `persistent=True`. What breaks?

Answer: They appear in `state_dict`. The exported model then has extra tensors (not exactly 19,272,192 values), and strict loading into a model without them fails.

Q131. Validation code calls `model.eval()` but never restores the mode. Why is that a problem even without dropout?

Answer: The caller's training loop would continue in eval mode. For modules whose behaviour depends on mode, training would silently change; the contract requires `model.train(was_training)`.

Q132. Why is ``mask.sum(-1) == 0 → raise`` the right check before masked softmax?

Answer: It detects any query row with no allowed key, which would produce an all $-\infty$ row and NaN probabilities. The spec requires failing clearly in that case.

Q133. `make_submission.sh` prints “tests/adapters.py still contains an unfinished adapter” even though all tests pass. What is the cause?

Answer: The script greps `adapters.py` for “NotImplementedError” or “TODO: connect your implementation”, and commented-out template lines still matched. Delete those leftover comment lines.